COSC494/594 Project 2 - Part A: Localization using Particle Filter

Instructor: Fei Liu fliu33@utk.edu
TA: Farong Wang fwang31@vols.utk.edu
University of Tennessee, Knoxville

Q1. Kinematic Car Motion Model

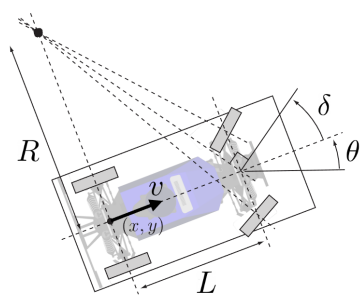

Figure 1: Visual representation of the kinematic car model

In this question, we derive and implement the kinematic bicycle model for a car-like robot. This model describes how the robot's state evolves over time given control inputs.

We assume the state is:

$$\mathbf{x} = (x, y, \theta)$$

and the control input is:

$$\mathbf{u} = (v, \delta)$$

where v is the linear velocity and δ is the steering angle.

Continuous-Time Model

Assuming constant velocity and steering angle over a short time interval, the kinematic model is:

$$\dot{x} = v \cos\theta$$
$$\dot{y} = v \sin\theta$$
$$\dot{\theta} = \omega$$

The angular velocity is given by:

$$\omega = \frac{v}{R} = \frac{v}{L} \tan\delta$$

Thus, the system becomes:

$$\frac{\partial x}{\partial t} = v \cos\theta, \quad \frac{\partial y}{\partial t} = v \sin\theta, \quad \frac{\partial \theta}{\partial t} = \frac{v}{L} \tan\delta$$

Discrete-Time Integration

We integrate over a time interval Δt assuming constant controls.

First, for orientation:

$$\int_{\theta_t}^{\theta_{t+1}} d\theta = \int_t^{t+\Delta t} \frac{v}{L} \tan\delta \, dt$$
$$\theta_{t+1} = \theta_t + \frac{v}{L} \tan\delta \, \Delta t$$

Next, for position:

$$x_{t+1} - x_t = \int_t^{t+\Delta t} v \cos\theta \, dt$$

Using change of variables:

$$dt = \frac{L}{v \tan\delta} d\theta$$

$$x_{t+1} - x_t = \frac{L}{\tan\delta} \int_{\theta_t}^{\theta_{t+1}} \cos\theta \, d\theta$$

$$x_{t+1} = x_t + \frac{L}{\tan\delta} \left[\sin\theta_{t+1} - \sin\theta_t\right]$$

Similarly for y:

$$y_{t+1} - y_t = \int_t^{t+\Delta t} v \sin\theta \, dt$$

$$y_{t+1} = y_t + \frac{L}{\tan\delta} \left[-\cos\theta_{t+1} + \cos\theta_t\right]$$

Final update equations:

$$\theta_{t+1} = \theta_t + \frac{v}{L} \tan\delta \, \Delta t$$
$$x_{t+1} = x_t + \frac{L}{\tan\delta} \left[\sin\theta_{t+1} - \sin\theta_t\right]$$
$$y_{t+1} = y_t + \frac{L}{\tan\delta} \left[-\cos\theta_{t+1} + \cos\theta_t\right]$$

Special Case (Zero Steering)

When $\delta = 0$, the model becomes singular due to division by $\tan\delta$.

In this case, the vehicle moves straight:

$$\theta_{t+1} = \theta_t$$
$$x_{t+1} = x_t + v \cos\theta \, \Delta t$$
$$y_{t+1} = y_t + v \sin\theta \, \Delta t$$

Implementation Note: Treat steering angles where $|\delta| <$ `delta_threshold` as zero.

Q1.1: Implementation of Motion Model

Implement the deterministic motion model in:

`src/proj2/proj2/motion_model.py`

Implementation:

```
# BEGIN QUESTION 1.1

    Please fill your code here.

# END QUESTION 1.1
```

- Implement `compute_changes(state, control, dt)`
- Handle both turning and straight motion cases
- Return $\Delta x, \Delta y, \Delta\theta$

Q1.2: Adding Noise to Motion Model

To make the model more realistic, we introduce noise:

1. Sample noisy controls:
$$\hat{v} \sim \mathcal{N}(v, \sigma_v^2), \quad \hat{\delta} \sim \mathcal{N}(\delta, \sigma_\delta^2)$$
2. Compute deterministic motion using noisy controls
3. Add output noise:
$$\Delta\mathbf{x} \sim \mathcal{N}(\Delta\hat{\mathbf{x}}, \Sigma)$$

Implementation:

```
# BEGIN QUESTION 1.2

    Please fill your code here.

# END QUESTION 1.2
```

- Implement `apply_motion_model`
- Use parameters:
  - `vel_std`, `delta_std`
  - `x_std`, `y_std`, `theta_std`
- Wrap θ into $(-\pi, \pi]$

Q1.3: Exploring the Motion Model Parameters

The noise in the motion model is controlled by the parameters σ_v, σ_δ (action noise) and $\sigma_x, \sigma_y, \sigma_\theta$ (model noise). Initial values are provided in:

`config/parameters.yaml`

You are encouraged to tune these parameters to better understand the physical behavior of the system. A visualization script is provided to help you analyze the effect of different noise settings:

```
python3  proj2/make_motion_model_plot.py
```

This script generates plots of sampled particle trajectories under the motion model. In this script, examine the `example_cases` dictionary. Each entry defines a tuple (v, δ, dt) representing nominal control inputs.

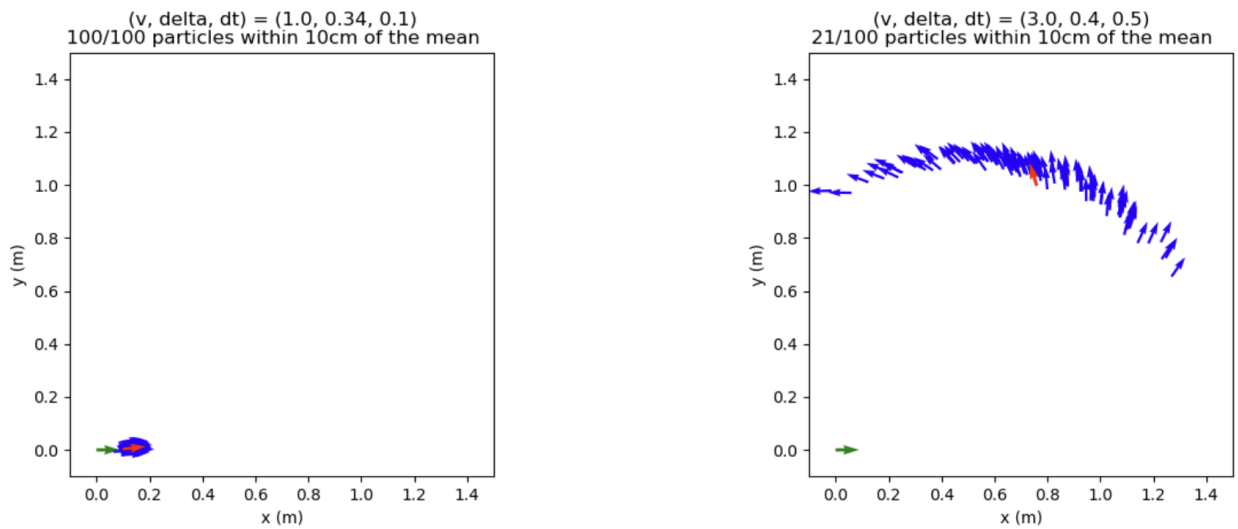

Figure 2: Left: The initial state (green) and state from integrating the noise-free model (red). Particles (blue) after moving forward and slightly to the left.
Right: Particles (blue) after moving forward and to the left. This banana-shaped curve is expected when noise is added to the system.

The Fig. 2 produces characteristic plots that reflect realistic vehicle motion. In particular:

- Low noise results in particles tightly clustered around the mean trajectory
- Higher velocity and steering introduce greater uncertainty and dispersion
- Turning motions produce a characteristic "banana-shaped" distribution

Parameter Tuning Task:

Tune your motion model parameters to match the qualitative behavior of the Fig. 2.

- Generate three plots using different parameter settings for:

$$(v, \delta, dt) = (3.0, 0.4, 0.5)$$

- Save your plots as:

`mm_noise1.png, mm_noise2.png, mm_noise3.png`

Write-up Requirements:

- Compare your plots with the reference behavior
- Explain how each parameter affects the spread and shape of the particle distribution
- Describe how you adjusted parameters to better match the expected motion

Note: Your final result does not need to exactly match the staff solution, but should demonstrate reasonable and interpretable behavior.

Q2. Sensor (Observation) Model

In this question, you will implement the LIDAR sensor model described in lecture. The mobile car's LIDAR unit emits infrared laser beams into the environment at fixed angular intervals and returns the measured distance along those beams. For an unsuccessful measurement, it may return `NaN` or 0. A sensor measurement $\mathbf{z}_t$ is a vector of distances, one for each beam:

$$\mathbf{z}_t = \left(z_t^1, \ldots, z_t^K\right)$$

Assuming that each distance z_t^k is conditionally independent (given the state $\mathbf{x}_t$ and map m), we can consider each beam separately.

First, we provide starter code to generate a simulated observation z_t^{k*} given the robot state $\mathbf{x}_t$. We use a raycasting library called `range_libc` (implemented in C++/CUDA), which casts rays into the map m from the robot state and measures the distance to any obstacle or wall encountered by the ray. This is similar to how the physical LIDAR unit measures distances.

Then, you will implement the sensor model

$$P(z_t^k \mid \mathbf{x}_t, m)$$

by considering the four different modes discussed in lecture. Specifically, you will implement a model that measures how likely it is to observe a real measurement z_t^k, given that the simulated observation is z_t^{k*}. You may find the lecture slides or the following summary useful. Note that the p_{short} factor should be modeled more simply here as a linear probability density function.

Sensor Model Modes

The sensor model is composed of four modes:

$$p_{\text{hit}}(z_t^k \mid \mathbf{x}_t, m) = \begin{cases} \mathcal{N}(z_t^k; z_t^{k*}, \sigma_{\text{hit}}^2) & 0 \le z_t^k \le z_{\max} \\ 0 & \text{otherwise} \end{cases}$$

$$p_{\text{short}}(z_t^k \mid \mathbf{x}_t, m) = \begin{cases} 2\dfrac{z_t^{k*} - z_t^k}{z_t^{k*}} & z_t^k < z_t^{k*} \\ 0 & \text{otherwise} \end{cases}$$

$$p_{\max}(z_t^k \mid \mathbf{x}_t, m) = \mathbb{I}(z_t^k = z_{\max})$$

$$p_{\text{rand}}(z_t^k \mid \mathbf{x}_t, m) = \begin{cases} \dfrac{1}{z_{\max}} & 0 \le z_t^k < z_{\max} \\ 0 & \text{otherwise} \end{cases}$$

The notation $\mathcal{N}(z_t^k; z_t^{k*}, \sigma_{\text{hit}}^2)$ means that we are evaluating the Gaussian probability density function at the point z_t^k, not sampling from a Gaussian. The density is

$$\frac{1}{\sqrt{2\pi\sigma_{\text{hit}}^2}} \exp\left(-\frac{1}{2}\left(\frac{z_t^k - z_t^{k*}}{\sigma_{\text{hit}}}\right)^2\right)$$

The final sensor model combines these modes using the weights

$$z_{\text{hit}}, \quad z_{\text{short}}, \quad z_{\max}, \quad z_{\text{rand}}$$

and the noise parameter σ_{hit}.

Q2.1 Precomputing the Sensor Model Table

You can speed up the sensor model by precomputing and caching the sensor probabilities. On the 2D occupancy grid map, continuous distances are discretized into pixel distances. You will store these probabilities in a table where:

- each row corresponds to an actual measured value
- each column corresponds to an expected measurement
- each table entry stores the corresponding probability

At runtime, you will use `range_libc` to generate simulated range measurements, then use the cached table to quickly look up the sensor probabilities.

Implement the sensor model in:

`src/proj2/proj2/sensor_model.py`

Inside the provided file, complete the following function:

`SingleBeamSensorModel.precompute_sensor_model`

Implementation:

```
# BEGIN QUESTION 2.1

    Please fill your code here.

# END QUESTION 2.1
```

- Implement the four sensor model modes:
  - `p_hit`
  - `p_short`
  - `p_max`
  - `p_rand`
- Combine them using the corresponding weights
- Accept zero as a valid weight for any component, as long as at least one weight is nonzero
- Normalize the table so that probabilities sum to 1 (by column)
- Use a vectorized NumPy implementation

Notes:

- The precomputed table should represent a proper probability distribution
- Be careful about edge cases such as $z_t^k = z_{\max}$ and invalid measurement ranges
- Since measurements are discretized, make sure your implementation is consistent with the table indexing convention used in the starter code

Q2.2: Visualizing and Tuning the Sensor Model

To better understand and tune the sensor model parameters, we provide tools to visualize the combined sensor likelihood for a single simulated observation.

Run the following script:

```
python3 proj2/make_sensor_model_single_plot.py
```

This script generates a plot of the conditional probability:

$$P(z_t^k \mid z_t^{k*})$$

for a fixed expected measurement z_t^{k*}.

Goal:

Use this visualization to understand how the different components of the sensor model contribute to the overall likelihood.

- p_{hit}: Gaussian peak around the expected measurement
- p_{short}: Higher probability for measurements shorter than expected
- $p_{\max}$: Spike at maximum range
- p_{rand}: Uniform distribution over valid ranges

Parameter Tuning:

- Adjust the weights:

  $$z_{\text{hit}}, \quad z_{\text{short}}, \quad z_{\max}, \quad z_{\text{rand}}$$

  Initial values are provided in:

  `config/parameters.yaml`

- Tune σ_{hit} to control the spread of the Gaussian
- Observe how each parameter changes the shape of the likelihood curve

Write-up Requirements:

- Include at least one plot generated from your tuned parameters
- Explain how each parameter affects the distribution
- Which component weight should be the largest?
- Why does this choice make sense physically for a real LIDAR sensor?